

Albertian Institute of Science & Technology

Lab Manual for

Database Management System Lab – 3136

Semester – III (CSE & ET)

Prepared By,

Roshni Jayan

Lecturer, Department of Computer Science & Engineering

INDEX

Exp No.	Name of Experiment	Date	Page No.	Faculty Signature
1	Installation And Introduction of SQL*Plus			
2	Implementation of Data Definition Language (DDL) Commands			
3	Implementation of Data Manipulation Language (DML) Commands			
4	Implementation of Constraints			
5	Computation on Tables using Built-in Functions			
6	Implementation of SQL Queries using WHERE, ORDER BY, DISTINCT, BETWEEN, IN and LIKE			
7	DDL Statements on Bank Database			
8	DDL & DML Statements on College Database			
9	DDL & DML Statements on Result Database			
10	Implementation of DML Commands – SELECT			
11	Implementation of Data Constraints on Purchase Database			
12	Implementation of Aggregate Functions in Bank Database			
13	Implementation of Nested Queries / Sub Queries			
14	Implementation of Inner Join			
15	Implementation of Book View			
16	Implementation of Stored Procedure			
17	Implementation of Stored Function			
18	Implementation of Cursor			
19	Implementation of Trigger			

EXPERIMENT NO: 1 **Date: _____**

INSTALLATION AND INTRODUCTION OF SQL*PLUS

AIM

To install Oracle Database software and familiarize with the SQL*Plus environment and basic SQL commands.

OBJECTIVES

- To install Oracle Database and SQL*Plus.
- To understand the SQL*Plus environment.
- To learn basic SQL*Plus commands.
- To connect to the Oracle Database using SQL*Plus.
- To execute simple SQL statements.
- To study the features and uses of SQL*Plus.

THEORY

SQL*Plus is a proprietary interactive and batch query command-line user interface provided by Oracle Corporation. It functions as an execution host environment that sends SQL statements and declarative PL/SQL programmatic blocks directly to the underlying Oracle Database engine for compilation and execution.

Beyond core standard SQL execution capabilities, SQL*Plus contains its own formatting commands (such as SET LINESIZE, COLUMN, and environment controls) which allow database administrators and developers to clean up query output layouts, execute local system script files, manage connections, and display data dictionary metadata definitions easily.

PROGRAM

```
-- 1. Establishing authentication session with administrative credentials
CONNECT system/oracle;

-- 2. Verifying the current active session user context
SHOW USER;

-- 3. Querying the current system date configuration from the default system pseudo-table
SELECT SYSDATE FROM DUAL;

-- 4. Retrieving the logical global database name string from the system catalog
SELECT NAME FROM V$DATABASE;

-- 5. Inspecting structural release version details of the active engine instance
SELECT * FROM V$VERSION;
```

```
-- 6. Terminating the client interface connection loop
EXIT;
```

OUTPUT

1. Session Connection Initiation

Connected.

2. User Context Check

USER is "SYSTEM"

3. Date Query Projection

SYSDATE

18-JUN-26

4. Database Schema Instance Label Verification

NAME

ORCL

5. Database Release Vector Version Inspection

BANNER

Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Production

6. Disconnection Confirmation

Disconnected from Oracle Database.

RESULT

Oracle SQL*Plus was successfully configured, and the foundational command execution layer was verified. Administrative queries tracking connection ownership context, date variables, and product database release parameters completed successfully without infrastructure exceptions.

EXPERIMENT NO: 2 **Date: _____**

IMPLEMENTATION OF DATA DEFINITION LANGUAGE (DDL) COMMANDS

AIM

To create a database and perform DDL commands such as CREATE, ALTER, RENAME, TRUNCATE, and DROP.

OBJECTIVES

- To create tables in the database.
- To modify table structure.
- To rename, truncate, and delete tables.

THEORY

Data Definition Language (DDL) statements form the structural design framework of a relational database. Instead of interacting with individual rows of data (which is handled by DML), DDL commands interact directly with the database's metadata blueprint.

- **CREATE**: Allocates space and constructs a brand new schema entity.
- **ALTER**: Modifies the structure of an existing object (e.g., adding or dropping a column).
- **RENAME**: Adjusts the identifier moniker mapping of a table in the system catalog.
- **TRUNCATE**: Instantly wipes all row entries out of a table by deallocating its storage segments. It operates much faster than a DELETE command because it doesn't log individual row purges.
- **DROP**: Wipes the entire table structure and its data completely out of the database memory space.

All DDL operations are **auto-committed**, meaning their updates are saved to the system catalog immediately and cannot be reversed using a ROLLBACK command.

PROGRAM

```
-- 1. Establishing the initial structural table layout
CREATE TABLE STUDENT (
    ROLLNO NUMBER(5) PRIMARY KEY,
    NAME VARCHAR2(30),
    MARK NUMBER(3)
);

-- 2. Viewing all tables currently mapped inside the user schema dictionary
SELECT TABLE_NAME FROM USER_TABLES;

-- 3. Reviewing the initial structural layout before modification
DESC STUDENT;

-- 4. Modifying the table footprint by appending a new column attribute
ALTER TABLE STUDENT ADD ADDRESS VARCHAR2(50);

-- 5. Checking the updated structural blueprint state
DESC STUDENT;

-- 6. Shifting the table entity mapping label name
RENAME STUDENT TO STUD;

-- 7. Confirming name changes across catalog references
SELECT TABLE_NAME FROM USER_TABLES;

-- 8. Wiping out data blocks via high-speed segment truncation
TRUNCATE TABLE STUD;

-- 9. Destructive removal of the entire table entity from the system dictionary
```

```
DROP TABLE STUD;
```

OUTPUT

1. Table Creation and Dictionary Verification

```
Table STUDENT created.

TABLE_NAME
------------------------------
STUDENT
```

Initial Structure of STUDENT (`DESC STUDENT;`):

```
Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ROLLNO                                    NOT NULL NUMBER(5)
 NAME                                               VARCHAR2(30)
 MARK                                               NUMBER(3)
```

2. Alteration Phase (Appending Columns)

```
Table altered.
```

Altered Structure of STUDENT (`DESC STUDENT;`):

```
Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ROLLNO                                    NOT NULL NUMBER(5)
 NAME                                               VARCHAR2(30)
 MARK                                               NUMBER(3)
 ADDRESS                                            VARCHAR2(50)
```

3. Table Modification and Purging Phases

```
Table renamed.

TABLE_NAME
------------------------------
STUDD

Table truncated.

Table dropped.
```

RESULT

Data Definition Language (DDL) statements such as CREATE, ALTER, RENAME, TRUNCATE, and DROP were executed successfully, allowing us to manage the structural framework lifecycle of the targeted database components without any errors.

EXPERIMENT NO: 3 **Date: _____**

IMPLEMENT DATA MANIPULATION LANGUAGE (DML) COMMANDS

AIM

To manipulate data in a database table using transactional modification parameters.

OBJECTIVES

1. To study the `INSERT` command using different structural syntax methods (implicit, explicit, and multi-row).
2. To study and execute targeted row modifications using the `UPDATE` command.
3. To study and execute row deletions using the conditional `DELETE` command.

THEORY

Data Manipulation Language (DML) statements manage data records within structural schema tables. Unlike Data Definition Language (DDL) commands, which alter table architecture instantly via auto-commit, DML commands execute within a temporary transaction workspace until saved or rolled back.

Methods of Data Insertion:

1. **Implicit Column Insertion:** Column values are provided sequentially without listing column headers. The values must match the exact table schema definition sequence:

   ```
   INSERT INTO STUDENT VALUES (101, 'ARUN', 85);
   ```

2. **Explicit Column Insertion:** Columns are declared explicitly inside a target array before passing values. This allows you to skip nullable columns or change insertion sequence safely:

   ```
   INSERT INTO STUDENT (ROLLNO, NAME, MARK) VALUES (102, 'ANU', 90);
   ```

PROGRAM

```
-- 1. Structural Table Creation (Ensuring the experiment is fully self-contained)

CREATE TABLE STUDENT (
    ROLLNO NUMBER(5) PRIMARY KEY,
    NAME VARCHAR2(30),
    MARK NUMBER(3)
);

-- 2. Data Population Method A: Standard Implicit Sequence

INSERT INTO STUDENT VALUES (101, 'ARUN', 85);

-- 3. Data Population Method B: Explicit Column Mapping

INSERT INTO STUDENT (ROLLNO, NAME, MARK) VALUES (102, 'ANU', 90);

-- 4. Checking table baseline state after populating records

SELECT * FROM STUDENT;

-- 5. Modifying specific values within an existing row conditionally

UPDATE STUDENT SET MARK = 95 WHERE ROLLNO = 101;

-- 6. Verifying modifications to ensure data consistency
```

```
SELECT * FROM STUDENT WHERE ROLLNO = 101;

-- 7. Removing specific row records from the dataset

DELETE FROM STUDENT WHERE ROLLNO = 102;

-- 8. Confirming current row states across active spaces

SELECT * FROM STUDENT;
```

OUTPUT

1. Baseline Architecture Setup

```
Table STUDENT created.
```

2. Evaluating Data Insertion Variations

```
1 row created. (Method A: Implicit columns successful)
1 row created. (Method B: Explicit columns successful)
```

State of Table After Insertion (`SELECT * FROM STUDENT;`):

```
    ROLLNO NAME                                MARK
---------- ------------------------------ ----------
       101 ARUN                                   85
       102 ANU                                    90
```

3. Record Value Modification

```
1 row updated.
```

State of Table After Update (`SELECT * FROM STUDENT WHERE ROLLNO = 101;`):

```
    ROLLNO NAME                                MARK
---------- ------------------------------ ----------
       101 ARUN                                   95
```

4. Row Removal Operations

```
1 row deleted. (ANU's record removed)
```

Final Verification State View (`SELECT * FROM STUDENT;`):

```
    ROLLNO NAME                                MARK
---------- ------------------------------ ----------
       101 ARUN                                   95
```

RESULT

Data Manipulation Language (DML) commands were executed successfully. The `STUDENT` table was created, and records were populated using diverse syntax entry structures. All subsequent updates and deletions processed accurately.

EXPERIMENT NO: 4 **Date: _____**

IMPLEMENT CONSTRAINTS

AIM

To implement integrity constraints in a table to guarantee operational entity constraints.

OBJECTIVES

- To study the use of PRIMARY KEY constraint.
- To study the use of NOT NULL constraint.
- To study the use of UNIQUE constraint.
- To study the use of CHECK constraint.
- To study the use of DEFAULT constraint.

THEORY

Integrity Constraints are a declarative set of business rules configured directly at the column or table level within a relational database. They act as automated system gatekeepers that filter incoming traffic, preventing invalid transaction entries or modifications from breaking underlying relational schemas.

- **`PRIMARY KEY`**: Uniquely identifies each row in a table. It implicitly combines a `UNIQUE` rule with a `NOT NULL` constraint, meaning it will never accept duplicate or empty values.
- **`NOT NULL`**: Ensures that a column cannot be left empty during an insertion or modification.
- **`UNIQUE`**: Restricts a column from accepting duplicate values across rows, ensuring that records remain distinct.
- **`CHECK`**: Evaluates a specific Boolean condition (e.g., assessing if a value is within a valid range). If the condition returns false, the transaction is rejected.
- **`DEFAULT`**: Automatically populates a pre-set value into a column if an explicit value is omitted during an insertion.

PROGRAM

```
-- 1. Table Creation utilizing standard declarative column constraints

CREATE TABLE EMPLOYEE (
    EMPID NUMBER(5) PRIMARY KEY,
    ENAME VARCHAR2(30) NOT NULL,
    EMAIL VARCHAR2(50) UNIQUE,
    SALARY NUMBER(8,2) CHECK (SALARY > 5000),
    CITY VARCHAR2(20) DEFAULT 'KOCHI'
);

-- 2. Valid Insertion: Checking fully-populated explicit rows

INSERT INTO EMPLOYEE VALUES (101, 'ANU', 'anu@gmail.com', 25000, 'TVM');

-- 3. Valid Insertion: Checking the DEFAULT constraint (Omit CITY to trigger
'KOCHI')

INSERT INTO EMPLOYEE (EMPID, ENAME, EMAIL, SALARY)
VALUES (102, 'ARUN', 'arun@gmail.com', 30000);

-- 4. Dynamic Verification: Reviewing active rows in the ledger

SELECT * FROM EMPLOYEE;

-- 5. Invalid Insertion: Triggering a PRIMARY KEY violation (Re-using EMPID 101)
INSERT INTO EMPLOYEE VALUES (101, 'RAVI', 'ravi@gmail.com', 20000, 'KOLLAM');
```

OUTPUT

1. Table Creation Phase

```
Table EMPLOYEE created.
```

2. Valid Record Population

```
1 row created. (Record for ANU added successfully)
1 row created. (Record for ARUN added; CITY defaults to 'KOCHI')
```

State of the Table Data Ledger (`SELECT * FROM EMPLOYEE;`):

```
     EMPID ENAME                 EMAIL              SALARY    CITY
---------- ----------------  ------------------ ---------- ------------------
       101 ANU                 anu@gmail.co          25000      TVM
       102 ARUN                arun@gmail.com        30000      KOCHI
```

3. Intercepting Constraint Violations

```
INSERT INTO EMPLOYEE VALUES (101, 'RAVI', 'ravi@gmail.com', 20000, 'KOLLAM')
*
ERROR at line 1:
ORA-00001: unique constraint (SYSTEM.SYS_C0011042) violated
Reason: EMPID '101' already exists in the table. The Primary Key constraint rejected
the duplicate transaction.
```

RESULT

The constraints PRIMARY KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT were implemented successfully, ensuring data integrity and consistency in the database.

EXPERIMENT NO: 5 **Date: _____**

COMPUTATION ON TABLES USING BUILT-IN FUNCTIONS

AIM

To perform computations using SQL built-in mathematical summary functions.

OBJECTIVES

- To study COUNT() function.
- To study SUM() function.
- To study AVG() function.
- To study MAX() function.
- To study MIN() function.

THEORY

Built-in Aggregate Functions (also known as group functions) perform mathematical computations on a collection of values across multiple rows to return a single, summary value.

- **COUNT(*)**: Calculates the total number of records matching the query criteria, including rows containing null parameters.
- **SUM(column)**: Adds up all the numeric values present inside a targeted attribute column range.
- **AVG(column)**: Computes the arithmetic mean of the column values. It automatically ignores null parameters during its calculation division step.
- **MAX(column) / MIN(column)**: Scan through the column datasets to isolate the absolute maximum and minimum scalar values within that partition domain.

PROGRAM

```
-- 1. Structuring the initial base data table

CREATE TABLE STUDENT (
    ROLLNO NUMBER(5) PRIMARY KEY,
    NAME VARCHAR2(30),
    MARK NUMBER(3)
);
-- 2. Populating baseline record entries

INSERT INTO STUDENT VALUES (1, 'ANU', 90);
INSERT INTO STUDENT VALUES (2, 'ARUN', 85);
INSERT INTO STUDENT VALUES (3, 'ATHIRA', 75);
INSERT INTO STUDENT VALUES (4, 'RAHUL', 65);
INSERT INTO STUDENT VALUES (5, 'MEERA', 95);

-- 3. Calculating the total number of student records present
SELECT COUNT(*) FROM STUDENT;

-- 4. Computing the cumulative sum of all marks
SELECT SUM(MARK) FROM STUDENT;

-- 5. Deriving the arithmetic average of student scores
SELECT AVG(MARK) FROM STUDENT;

-- 6. Locating the highest score recorded in the class
SELECT MAX(MARK) FROM STUDENT;

-- 7. Locating the lowest score recorded in the class
SELECT MIN(MARK) FROM STUDENT;
```

OUTPUT

1. Table Creation & Initial Record Population

```
Table STUDENT created.

1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
```

2. Execution of Built-In Aggregate Functions

Row Count Tracking:

```
  COUNT(*)
----------
         5
```

Cumulative Score Addition:

```
SUM(MARK)
----------
       410
```

Class Average Computation:

```
AVG(MARK)
----------
        82
```

Peak Value Verification:

```
MAX(MARK)
----------
        95
```

Floor Value Verification:

```
MIN(MARK)
----------
        65
```

RESULT

Built-in functions such as COUNT(), SUM(), AVG(), MAX() and MIN() were executed successfully.

EXPERIMENT NO: 6 **Date: _____**

IMPLEMENTATION OF SQL QUERIES USING WHERE, ORDER BY, DISTINCT, BETWEEN, IN AND LIKE

AIM

To retrieve records from a table using various SQL clauses and conditional filter operators.

OBJECTIVES

1. To study and implement the data filtering rules of the `WHERE` clause.
2. To study and arrange structural records chronologically using the `ORDER BY` sorting direction.
3. To study and eliminate redundant duplicate parameters using `DISTINCT`.
4. To study range boundary mapping with the inclusive `BETWEEN` operator.
5. To study discrete array set comparison logic filters using the `IN` operator.
6. To search for structural substring pattern sequences using wildcards with `LIKE`.

THEORY

Conditional database lookup configurations allow users to isolate specific data points from large background tables.

- **WHERE**: Applies conditional logic constraints to evaluate true/false conditions on a row-by-row level.
- **ORDER BY**: Modifies the output representation sorting alignment into ascending (`ASC`, default) or descending (`DESC`) orientation sequences.
- **DISTINCT**: Runs an inline distinct filter that drops identical repeating data parameters, outputting unique elements.
- **BETWEEN**: Performs a boundary scanning operation across an inclusive lower and upper numeric/date range limit.
- **IN**: Validates membership loops across an explicit, discrete set array listing.
- **LIKE**: Utilizes wildcard character constraints, where the percent symbol (%) matches any sequence of zero or more string characters.

PROGRAM

```
-- 1. Filtering records where MARK is strictly greater than 80

SELECT * FROM STUDENT WHERE MARK > 80;

-- 2. Sorting records by MARK in descending numerical order

SELECT * FROM STUDENT ORDER BY MARK DESC;

-- 3. Displaying unique courses available inside the table matrix

SELECT DISTINCT COURSE FROM STUDENT;

-- 4. Filtering records with MARKS within an inclusive range of 70 and 90

SELECT * FROM STUDENT WHERE MARK BETWEEN 70 AND 90;

-- 5. Filtering records where the COURSE matches values inside a discrete set array

SELECT * FROM STUDENT WHERE COURSE IN ('CS', 'EC');

-- 6. Filtering records where NAME begins with the string character 'A'

SELECT * FROM STUDENT WHERE NAME LIKE 'A%';
```

OUTPUT

1. Filtered by Mark (> 80)

```
+--------+--------+--------+------+-------+
| ROLLNO | NAME   | COURSE | MARK | PHNO  |
+--------+--------+--------+------+-------+
|    101 | ARJUN  | CS     |   92 | 98765 |
|    104 | ANANYA | EC     |   88 | 94471 |
+--------+--------+--------+------+-------+
2 rows in set (0.00 sec)
```

2. Sorted by Mark (Descending Order)

```
+--------+--------+--------+------+-------+
| ROLLNO | NAME   | COURSE | MARK | PHNO  |
+--------+--------+--------+------+-------+
|    101 | ARJUN  | CS     |   92 | 98765 |
|    104 | ANANYA | EC     |   88 | 94471 |
|    103 | BINOY  | ME     |   75 | 96321 |
|    102 | DEV    | CS     |   64 | 91123 |
+--------+--------+--------+------+-------+
4 rows in set (0.00 sec)
```

3. Distinct Courses Listing

```
+--------+
| COURSE |
+--------+
| CS     |
| ME     |
| EC     |
+--------+
3 rows in set (0.01 sec)
```

4. Filtered by Marks Range (BETWEEN 70 AND 90)

```
+--------+--------+--------+------+-------+
| ROLLNO | NAME   | COURSE | MARK | PHNO  |
+--------+--------+--------+------+-------+
|    103 | BINOY  | ME     |   75 | 96321 |
|    104 | ANANYA | EC     |   88 | 94471 |
+--------+--------+--------+------+-------+
2 rows in set (0.00 sec)
```

5. Filtered by Course Set (IN ('CS','EC'))

```
+--------+--------+--------+------+-------+
| ROLLNO | NAME   | COURSE | MARK | PHNO  |
+--------+--------+--------+------+-------+
|    101 | ARJUN  | CS     |   92 | 98765 |
|    102 | DEV    | CS     |   64 | 91123 |
|    104 | ANANYA | EC     |   88 | 94471 |
+--------+--------+--------+------+-------+
3 rows in set (0.00 sec)
```

6. Pattern Matched by Name (LIKE 'A%')

```
+--------+--------+--------+------+-------+
| ROLLNO | NAME   | COURSE | MARK | PHNO  |
+--------+--------+--------+------+-------+
|    101 | ARJUN  | CS     |   92 | 98765 |
|    104 | ANANYA | EC     |   88 | 94471 |
+--------+--------+--------+------+-------+
2 rows in set (0.00 sec)
```

RESULT

SQL queries using WHERE, ORDER BY, DISTINCT, BETWEEN, IN and LIKE clauses were executed successfully and the required records were retrieved from the table.

EXPERIMENT NO: 7 **Date: _____**

DDL STATEMENTS ON BANK DATABASE

AIM

To implement DDL (Data Definition Language) statements on a Bank Database structure configuration.

OBJECTIVES

1. Add the column `EMAIL` to the `CUSTOMER` table.
2. Add a new column `BANK_MANAGER` to the `BANK_BRN` table.
3. Delete the column `BANK_MANAGER` from `BANK_BRN` and rename the column `ID` to `CUSID` in the `CUSTOMER` table.
4. Delete the table `BANK_BRN`.

PROGRAM

```
-- 1. Initial configuration of structural layout tables

CREATE TABLE CUSTOMER (
    ID NUMBER(5) PRIMARY KEY,
    NAME VARCHAR2(30) NOT NULL,
    ADDRESS VARCHAR2(50),
    PHONE VARCHAR2(15)
);

CREATE TABLE BANK_BRN (
    BRANCH_ID NUMBER(5) PRIMARY KEY,
    BRANCH_NAME VARCHAR2(30),
    CITY VARCHAR2(30)
);

-- 2. Inspecting table structures and baseline layout entries
DESC CUSTOMER;
SELECT TABLE_NAME FROM USER_TABLES;

-- 3. Appending specific data field columns via ALTER modifications
ALTER TABLE CUSTOMER ADD EMAIL VARCHAR2(40);
ALTER TABLE BANK_BRN ADD BANK_MANAGER VARCHAR2(30);

-- 4. Purging specific targeted structural layout elements
ALTER TABLE BANK_BRN DROP COLUMN BANK_MANAGER;

-- 5. Shifting specific column identifiers
ALTER TABLE CUSTOMER RENAME COLUMN ID TO CUSID;

-- 6. Removing target structural components from system dictionary
DROP TABLE BANK_BRN;
```

OUTPUT

1. Table Creation & Initial Baseline Inspection

```
Table CUSTOMER created.
Table BANK_BRN created.
```

Structure of CUSTOMER (`DESC CUSTOMER;`):

```
Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ID                                        NOT NULL NUMBER(5)
 NAME                                      NOT NULL VARCHAR2(30)
```

```
 ADDRESS                                                 VARCHAR2(50)
 PHONE                                                   VARCHAR2(15)
```

Active Schema Tables (`SELECT TABLE_NAME...`):

```
TABLE_NAME
------------------------------
CUSTOMER
BANK_BRN
```

2. Appending Attributes (`EMAIL` & `BANK_MANAGER`)

```
Table CUSTOMER altered (Column EMAIL added).
Table BANK_BRN altered (Column BANK_MANAGER added).
```

Updated Structure of CUSTOMER:

```
Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ID                                        NOT NULL NUMBER(5)
 NAME                                      NOT NULL VARCHAR2(30)
 ADDRESS                                            VARCHAR2(50)
 PHONE                                              VARCHAR2(15)
 EMAIL                                              VARCHAR2(40)
```

3. Dropping and Renaming Structural Elements

```
Table BANK_BRN altered (Column BANK_MANAGER dropped).
Table CUSTOMER altered (Column ID renamed to CUSID).
```

Final Dynamic Structure of CUSTOMER:

```
Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 CUSID                                     NOT NULL NUMBER(5)
 NAME                                      NOT NULL VARCHAR2(30)
 ADDRESS                                            VARCHAR2(50)
 PHONE                                              VARCHAR2(15)
 EMAIL                                              VARCHAR2(40)
```

4. Destructive Purge of Schema Objects

```
Table BANK_BRN dropped.
```

RESULT

DDL statements such as CREATE, ALTER, DESC, SHOW and DROP were executed successfully on the Bank Database.

EXPERIMENT NO: 8 **Date: _____**

DDL & DML STATEMENTS ON COLLEGE DATABASE

AIM

To implement structural definition operations targeting an academic informational model configuration layer.

OBJECTIVES

1. Create the database tables with the necessary structural constraints.
2. Add a new attribute `PHNO` to the `STUDENT` table.
3. Add a new attribute `GUARDIAN` to the `STUDENT` table.
4. Change the attribute name `GUARDIAN` to `PARENT` in the `STUDENT` table.
5. Write a query to change the table name from `STUDENT` to `STUDENT_DETAILS`.

PROGRAM

```
-- 1. Initial creation of the STUDENT table schema structure
CREATE TABLE STUDENT (
    ROLLNO NUMBER(5) PRIMARY KEY,
    NAME VARCHAR2(30) NOT NULL,
    COURSE VARCHAR2(20),
    MARK NUMBER(3)
);

-- 2. Appending a new attribute PHNO to the table layout
ALTER TABLE STUDENT ADD PHNO VARCHAR2(15);

-- 3. Appending a new attribute GUARDIAN to the table layout
ALTER TABLE STUDENT ADD GUARDIAN VARCHAR2(30);

-- 4. Modifying attribute identifier metadata from GUARDIAN to PARENT
ALTER TABLE STUDENT RENAME COLUMN GUARDIAN TO PARENT;

-- 5. Transitioning entity moniker designation from STUDENT to STUDENT_DETAILS
RENAME STUDENT TO STUDENT_DETAILS;
```

OUTPUT

1. Initial Table Creation & Status View

```
Table STUDENT created.

TABLE_NAME
------------------------------
STUDENT
```

Initial Structure of STUDENT (`DESC STUDENT;`):

```
Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ROLLNO                                    NOT NULL NUMBER(5)
 NAME                                      NOT NULL VARCHAR2(30)
 COURSE                                             VARCHAR2(20)
 MARK                                               NUMBER(3)
```

2. Appending Attributes (`PHNO` & `GUARDIAN`)

```
Table STUDENT altered.
Table STUDENT altered.
```

Updated Structure with New Structural Leaves:

```
Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ROLLNO                                    NOT NULL NUMBER(5)
 NAME                                      NOT NULL VARCHAR2(30)
 COURSE                                             VARCHAR2(20)
 MARK                                               NUMBER(3)
 PHNO                                               VARCHAR2(15)
 GUARDIAN                                           VARCHAR2(30)
```

3. Column Identifier Renamed (GUARDIAN - PARENT)

```
Table STUDENT altered.
```

Structural Framework State After Column Rename:

```
Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ROLLNO                                    NOT NULL NUMBER(5)
 NAME                                      NOT NULL VARCHAR2(30)
 COURSE                                             VARCHAR2(20)
 MARK                                               NUMBER(3)
 PHNO                                               VARCHAR2(15)
 PARENT                                             VARCHAR2(30)
```

4. Relocating Table Name Mapping (STUDENT - STUDENT_DETAILS)

```
Table renamed.

TABLE_NAME
------------------------------
STUDENT_DETAILS
```

RESULT

DDL statements such as CREATE, SHOW and ALTER were executed successfully on the College Database, and the required modifications were performed on the table.

EXPERIMENT NO: 9 **Date:** _____

DDL & DML STATEMENTS ON RESULT DATABASE

AIM

To manage evaluation records using mathematical manipulation logic configurations.

OBJECTIVES

1. Create the `MARKLIST` table with required fields and constraints.
2. Insert sample student performance records into the table.
3. Display all structural records from the table.
4. Update missing mark entries containing NULL values.
5. Calculate and populate cumulative academic data into the `TOTAL` column.
6. Retrieve a filtered projection displaying `REGNO`, `NAME`, and `TOTAL` marks.
7. Identify students who scored above 90 in Physics.
8. Identify students whose cumulative total mark exceeds 250.
9. Filter and isolate students who passed in all individual subjects (assuming a passing mark threshold of 50%).
10. Isolate elite performance profiles matching students scoring above 90 across all subjects.
11. Identify risk-category student profiles who failed in any individual subject.
12. List student records whose cumulative score falls within a specific percentage boundary (between 70% and 80%).
13. Retrieve targeted full profile details for the student named 'AKHIL'.
14. Modify and update 'AKHIL's individual mathematical score parameter to 86.

PROGRAM

1. Create table marklist with required fields.

```
CREATE TABLE MARKLIST (
    REGNO NUMBER(5) PRIMARY KEY,
    NAME VARCHAR2(50),
    PHYSICS NUMBER(3),
    CHEMISTRY NUMBER(3),
    MATHS NUMBER(3),
    TOTAL NUMBER(4)
);
```

2. Insert records into the table.

```
INSERT INTO MARKLIST (REGNO, NAME, PHYSICS, CHEMISTRY, MATHS) VALUES (101, 'AKHIL',
95, 88, NULL);
INSERT INTO MARKLIST (REGNO, NAME, PHYSICS, CHEMISTRY, MATHS) VALUES (102, 'ANU',
78, 82, 85);
INSERT INTO MARKLIST (REGNO, NAME, PHYSICS, CHEMISTRY, MATHS) VALUES (103, 'ROHIT',
45, 55, 62);
INSERT INTO MARKLIST (REGNO, NAME, PHYSICS, CHEMISTRY, MATHS) VALUES (104, 'SNEHA',
92, 94, 91);
```

3. Display all records in the table.

```
SELECT * FROM MARKLIST;
```

4. Update missing marks.

```
UPDATE MARKLIST SET MATHS = 84 WHERE REGNO = 101;
```

5. Calculate and update total marks.

```
UPDATE MARKLIST SET TOTAL = PHYSICS + CHEMISTRY + MATHS;
```

6. Display regno, name and total marks.

```
SELECT REGNO, NAME, TOTAL FROM MARKLIST;
```

7. Find students who scored above 90 in Physics.

```
SELECT * FROM MARKLIST WHERE PHYSICS > 90;
```

8. Find students whose total mark is above 250.

```
SELECT * FROM MARKLIST WHERE TOTAL > 250;
```

9. Find students who passed in all subjects.

```
SELECT * FROM MARKLIST WHERE PHYSICS >= 50 AND CHEMISTRY >= 50 AND MATHS >= 50;
```

10. Find students who scored above 90 in all subjects.

```
SELECT * FROM MARKLIST WHERE PHYSICS > 90 AND CHEMISTRY > 90 AND MATHS > 90;
```

11. Find students who failed in any subject.

```
SELECT * FROM MARKLIST WHERE PHYSICS < 50 OR CHEMISTRY < 50 OR MATHS < 50;
```

12. List students whose total mark is between 70% and 80%.

```
SELECT * FROM MARKLIST WHERE TOTAL BETWEEN 210 AND 240;
```

13. Find details of student Akhil.

```
SELECT * FROM MARKLIST WHERE NAME = 'AKHIL';
```

14. Update Akhil's Maths mark to 86.

```
UPDATE MARKLIST SET MATHS = 86 WHERE NAME = 'AKHIL';
UPDATE MARKLIST SET TOTAL = PHYSICS + CHEMISTRY + MATHS WHERE NAME = 'AKHIL';
```

OUTPUT

1. Create table marklist with required fields.

Plaintext

```
Table MARKLIST created.
```

2. Insert records into the table.

Plaintext

```
1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.
```

3. Display all records in the table.

Plaintext

```
REGNO NAME       PHYSICS CHEMISTRY     MATHS     TOTAL
----- ---------- ------- --------- --------- ---------
  101 AKHIL           95        88      NULL      NULL
  102 ANU             78        82        85      NULL
  103 ROHIT           45        55        62      NULL
  104 SNEHA           92        94        91      NULL
```

4. Update missing marks.

Plaintext

```
1 row updated.
```

5. Calculate and update total marks.

Plaintext

```
4 rows updated.
```

6. Display regno, name and total marks.

Plaintext

```
REGNO NAME            TOTAL
----- ---------- ----------
  101 AKHIL             267
  102 ANU               245
  103 ROHIT             162
  104 SNEHA             277
```

7. Find students who scored above 90 in Physics.

Plaintext

```
REGNO NAME       PHYSICS CHEMISTRY     MATHS     TOTAL
----- ---------- ------- --------- --------- ---------
  101 AKHIL           95        88        84       267
  104 SNEHA           92        94        91       277
```

8. Find students whose total mark is above 250.

Plaintext

```
REGNO NAME       PHYSICS CHEMISTRY     MATHS     TOTAL
----- ---------- ------- --------- --------- ---------
  101 AKHIL           95        88        84       267
  104 SNEHA           92        94        91       277
```

9. Find students who passed in all subjects.

Plaintext

```
REGNO NAME       PHYSICS CHEMISTRY     MATHS     TOTAL
----- ---------- ------- --------- --------- ---------
  101 AKHIL           95        88        84       267
  102 ANU             78        82        85       245
  104 SNEHA           92        94        91       277
```

10. Find students who scored above 90 in all subjects.

Plaintext

```
REGNO NAME       PHYSICS CHEMISTRY     MATHS     TOTAL
----- ---------- ------- --------- --------- ---------
  104 SNEHA           92        94        91       277
```

11. Find students who failed in any subject.

Plaintext

```
REGNO NAME       PHYSICS CHEMISTRY     MATHS     TOTAL
----- ---------- ------- --------- --------- ---------
  103 ROHIT           45        55        62       162
```

12. List students whose total mark is between 70% and 80%.

Plaintext

```
No rows selected.
```

13. Find details of student Akhil.

Plaintext

```
REGNO NAME       PHYSICS CHEMISTRY     MATHS     TOTAL
----- ---------- ------- --------- --------- ---------
  101 AKHIL           95        88        84       267
```

14. Update Akhil's Maths mark to 86.

Plaintext

```
1 row updated.
1 row updated.
```

RESULT

DDL and DML statements such as CREATE, INSERT, UPDATE and SELECT were executed successfully on the Result Database, and the required queries were performed successfully.

EXPERIMENT NO: 10 **Date: _____**

IMPLEMENTATION OF DML COMMANDS – SELECT

AIM

To execute granular relational filtering using substring expression lookups.

OBJECTIVES

1. Create the `STUDENT` table with required fields and constraints.
2. Insert 5 sample structural records into the table.
3. Find the details of students in branch 'CS' whose name starts with 'A'/'a'.
4. Display `STD_ID`, `NAME`, and `BRANCH` of students whose name contains 'kh' and place is 'Pala'.
5. Find the name and place of students from 'Ernakulam' district whose name ends with 'n'.
6. Find the details of students whose second letter in the name is 'b' and ends with 'n'.
7. Display `STD_ID`, `NAME`, and `BRANCH` of students whose third letter is 's' and fifth letter is 'l'.
8. Display `STD_ID`, `NAME`, and `PLACE` of students whose place has the letter 'l' in it.
9. Find the details of students in the 'EC' branch whose name starts with 'al'.
10. Find the details of students in the 'CS' branch whose name starts with 'k' and contains the string 'vi' in between.
11. Find the name and place of students assigned to batch 'A'/'a' across all branches.

PROGRAM

```
-- 1. Create table student with required fields.
CREATE TABLE STUDENT (
    STD_ID NUMBER(5) PRIMARY KEY,
    NAME VARCHAR2(50) NOT NULL,
    BRANCH VARCHAR2(10),
    PLACE VARCHAR2(30),
    BATCH VARCHAR2(5)
);

-- 2. Insert 5 records into the table.
INSERT INTO STUDENT VALUES (201, 'AKHIL', 'CS', 'Pala', 'A');
INSERT INTO STUDENT VALUES (202, 'ABIN', 'EC', 'Ernakulam', 'B');
INSERT INTO STUDENT VALUES (203, 'ANSON', 'ME', 'Kottayam', 'A');
INSERT INTO STUDENT VALUES (204, 'ALAN', 'EC', 'Calicut', 'A');
INSERT INTO STUDENT VALUES (205, 'KAVIN', 'CS', 'Pala', 'B');

-- 3. Find details of student in branch 'CS' whose name starts with 'a'.
SELECT * FROM STUDENT WHERE BRANCH='CS' AND NAME LIKE 'A%';

-- 4. Display std_id, name and branch of student whose name contains 'kh' and place
is 'Pala'.
SELECT STD_ID, NAME, BRANCH FROM STUDENT WHERE NAME LIKE '%KH%' AND PLACE='Pala';

-- 5. Find name and place of student from Ernakulam district whose name ends with
'n'.
SELECT NAME, PLACE FROM STUDENT WHERE PLACE='Ernakulam' AND NAME LIKE '%N';

-- 6. Find details of student whose second letter in the name is 'b' and ends with
'n'.
SELECT * FROM STUDENT WHERE NAME LIKE '_B%N';

-- 7. Display std_id, name and branch of student whose third letter is 's' and fifth
letter is 'l'.
SELECT STD_ID, NAME, BRANCH FROM STUDENT WHERE NAME LIKE '__S_L%';

-- 8. Display std_id, name and place of student whose place has letter 'l' in it.
SELECT STD_ID, NAME, PLACE FROM STUDENT WHERE PLACE LIKE '%L%';
```

```
-- 9. Find details of student in EC branch whose name starts with 'al'.
SELECT * FROM STUDENT WHERE BRANCH='EC' AND NAME LIKE 'AL%';

-- 10. Find details of student in CS branch whose name starts with 'k' and has
string 'vi' in between.
SELECT * FROM STUDENT WHERE BRANCH='CS' AND NAME LIKE 'K%VI%';

-- 11. Find name and place of students in batch 'a' of all branches.
SELECT NAME, PLACE FROM STUDENT WHERE BATCH='A';
```

OUTPUT

1. Create table student with required fields.

```
Table STUDENT created.
```

2. Insert 5 records into the table.

```
1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.
```

3. Find details of student in branch 'CS' whose name starts with 'a'.

```
STD_ID          NAME                 BRANCH     PLACE      BATCH
--------- -------------------- ---------- ---------- -----
    201         AKHIL                   CS         Pala       A
```

4. Display std_id, name and branch of student whose name contains 'kh' and place is 'Pala'.

Plaintext

```
    STD_ID NAME        BRANCH
---------- ---------- ----------
       201 AKHIL        CS
```

5. Find name and place of student from Ernakulam district whose name ends with 'n'.

```
NAME          PLACE
---------- -------------
ABIN        Ernakulam
```

6. Find details of student whose second letter in the name is 'b' and ends with 'n'.

Plaintext

```
    STD_ID NAME            BRANCH     PLACE           BATCH
---------- -------------- ---------- --------------- -----
       202 ABIN            EC         Ernakulam         B
```

7. Display std_id, name and branch of student whose third letter is 's' and fifth letter is 'l'.

Plaintext

```
    STD_ID NAME            BRANCH
---------- -------------- ----------
       203 ANSON            ME
```

8. Display std_id, name and place of student whose place has letter 'l' in it.

Plaintext

```
    STD_ID NAME           PLACE
---------- ------------ -------------
       201 AKHIL         Pala
       202 ABIN          Ernakulam
       204 ALAN          Calicut
```

9. Find details of student in EC branch whose name starts with 'al'.

STD_ID	NAME	BRANCH	PLACE	BATCH
204	ALAN	EC	Calicut	A

10. Find details of student in CS branch whose name starts with 'k' and has string 'vi' in between.

STD_ID	NAME	BRANCH	PLACE	BATCH
205	KAVIN	CS	Pala	B

11. Find name and place of students in batch 'a' of all branches.

NAME	PLACE
AKHIL	Pala
ANSON	Kottayam
ALAN	Calicut

RESULT

SELECT queries using conditions and LIKE operators were executed successfully and the required records were retrieved from the STUDENT table.

EXPERIMENT NO: 11 **Date: _____**

IMPLEMENTATION OF DATA CONSTRAINTS ON PURCHASE DATABASE

AIM

To construct operational parent-child relational mappings secured via foreign key check limits.

OBJECTIVES

1. Create table **ITEM** with fields **item_code, item_name, unit_price** satisfying the following conditions:
   - item_code should not be NULL and should be PRIMARY KEY.
   - item_name should not be NULL.
   - unit_price should be greater than 10.
2. Create table **PURCHASE** with fields **invoiceno, item_code, quantity** satisfying the following conditions:
   - invoiceno should not be NULL.
   - item_code should not be NULL.
   - quantity must be greater than 10.
3. Insert data into tables and check that all constraints are working properly.

PROGRAM

```
-- 1. Create parent table ITEM with inline column constraints

CREATE TABLE ITEM (
    ITEM_CODE NUMBER(5) PRIMARY KEY,
    ITEM_NAME VARCHAR2(30) NOT NULL,
    UNIT_PRICE NUMBER(8,2) CHECK (UNIT_PRICE > 10)
);

-- 2. Create child table PURCHASE with referential constraints

CREATE TABLE PURCHASE (
    INVOICENO NUMBER(5) NOT NULL,
    ITEM_CODE NUMBER(5) NOT NULL,
    QUANTITY NUMBER(5) CHECK (QUANTITY > 10),
    FOREIGN KEY (ITEM_CODE) REFERENCES ITEM (ITEM_CODE)
);

-- 3. Verifying standard successful insertions (Valid Data)

INSERT INTO ITEM VALUES (501, 'Laptops', 45000.00);
INSERT INTO PURCHASE VALUES (1001, 501, 15);

-- 4. Verifying CHECK Constraint Violation (Price <= 10)

INSERT INTO ITEM VALUES (502, 'Ballpoint Pen', 5.00);

-- 5. Verifying CHECK Constraint Violation (Quantity <= 10)

INSERT INTO PURCHASE VALUES (1002, 501, 4);

-- 6. Verifying REFERENTIAL Integrity Violation (Orphan Foreign Key)

INSERT INTO PURCHASE VALUES (1003, 999, 20);
```

OUTPUT

1. Table Creation

```
Table ITEM created.
Table PURCHASE created.
```

2. Valid Data Insertion (Constraints Satisfied)

```
1 row inserted into ITEM (501, 'Laptops', 45000.00).
1 row inserted into PURCHASE (1001, 501, 15).
```

3. Intercepting CHECK Constraint Violation on Parent Table (ITEM)

```
INSERT INTO ITEM VALUES (502, 'Ballpoint Pen', 5.00)
*
ERROR at line 1:
ORA-02290: check constraint (SYS_C007542) violated
Reason: UNIT_PRICE (5.00) is not greater than 10.
```

4. Intercepting CHECK Constraint Violation on Child Table (PURCHASE)

```
INSERT INTO PURCHASE VALUES (1002, 501, 4)
*
ERROR at line 1:
ORA-02290: check constraint (SYS_C007545) violated
Reason: QUANTITY (4) is not greater than 10.
```

5. Intercepting Referential Integrity Violation (FOREIGN KEY)

```
INSERT INTO PURCHASE VALUES (1003, 999, 20)
*
ERROR at line 1:
ORA-02291: integrity constraint (SYS_C007546) violated - parent key not found
Reason: ITEM_CODE '999' does not exist in parent table ITEM.
```

RESULT

Data constraints such as PRIMARY KEY, NOT NULL, CHECK, and FOREIGN KEY were implemented successfully, and constraint violations were detected correctly during data insertion.

EXPERIMENT NO: 12 **Date: _____**

IMPLEMENTATION OF AGGREGATE FUNCTIONS IN BANK DATABASE

AIM

To implement Aggregate Functions in a Bank Database using multi-column grouping logic matrices.

OBJECTIVES

1. Create table `account_details` with fields: `acc_no`, `holder_name`, `branch` (Kottayam / Pala / Trivandrum), `acc_type` (Savings / Current), and `balance`.
2. Insert 5 records into the table (ensuring at least one holder has multiple accounts to test all constraints).
3. Find the average balance of the bank.
4. Find the average balance of each type of account.
5. Find the average balance of Pala branch for each type of account.
6. Find the minimum balance for each branch in each type of account.
7. Find the total number of branches of the bank.
8. Find the total number of accounts in the bank.
9. Find the maximum balance of Savings account.
10. Find the branch having maximum balance.
11. Find the total deposit in each branch.
12. Find the total deposit in Current account in Kottayam branch.
13. Find the person having more than one account in the bank.

PROGRAM

```
-- 1. Table Creation with domain constraints

CREATE TABLE account_details (
    acc_no INT PRIMARY KEY,
    holder_name VARCHAR(50),
    branch VARCHAR(20) CHECK (branch IN ('Kottayam', 'Pala', 'Trivandrum')),
    acc_type VARCHAR(15) CHECK (acc_type IN ('Savings', 'Current')),
    balance DECIMAL(10, 2)
);

-- 2. Insert 5 Records with a duplicate holder to test constraints

INSERT INTO account_details VALUES (101, 'Arjun Das', 'Kottayam', 'Savings',
50000.00);
INSERT INTO account_details VALUES (102, 'Meera Nair', 'Pala', 'Current', 75000.00);
INSERT INTO account_details VALUES (103, 'Arjun Das', 'Pala', 'Savings', 25000.00);
INSERT INTO account_details VALUES (104, 'Rahul Ravi', 'Trivandrum', 'Savings',
12000.00);
INSERT INTO account_details VALUES (105, 'Sara John', 'Kottayam', 'Current',
90000.00);

-- 3. Find the average balance of the bank

SELECT AVG(balance) AS bank_average_balance FROM account_details;

-- 4. Find the average balance of each type of account

SELECT acc_type, AVG(balance) AS avg_balance FROM account_details GROUP BY acc_type;

-- 5. Find the average balance of Pala branch for each type of account

SELECT acc_type, AVG(balance) AS pala_avg_balance FROM account_details WHERE branch
= 'Pala' GROUP BY acc_type;
```

```
-- 6. Find the minimum balance for each branch in each type of account

SELECT branch, acc_type, MIN(balance) AS min_balance FROM account_details GROUP BY
branch, acc_type;

-- 7. Find the total number of distinct branches of the bank

SELECT COUNT(DISTINCT branch) AS total_branches FROM account_details;

-- 8. Find the total number of accounts in the bank

SELECT COUNT(acc_no) AS total_accounts FROM account_details;

-- 9. Find the maximum balance of Savings account

SELECT MAX(balance) AS max_savings_balance FROM account_details WHERE acc_type =
'Savings';

-- 10. Find the branch having maximum balance

SELECT branch, MAX(balance) AS max_balance FROM account_details GROUP BY branch
ORDER BY max_balance DESC LIMIT 1;

-- 11. Find the total deposit in each branch

SELECT branch, SUM(balance) AS total_deposit FROM account_details GROUP BY branch;

-- 12. Find the total deposit in Current account in Kottayam branch

SELECT SUM(balance) AS kottayam_current_total FROM account_details WHERE branch =
'Kottayam' AND acc_type = 'Current';

-- 13. Find the person having more than one account in the bank

SELECT holder_name, COUNT(*) AS account_count FROM account_details GROUP BY
holder_name HAVING COUNT(*) > 1;
```

OUTPUT

1. Table Creation & Base Entry Population

```
Table account_details created.
```

2. Query Objective Results

```
1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.
```

3. Average Balance of the Bank:

```
+----------------------+
| bank_average_balance |
+----------------------+
|             50400.00 |
+----------------------+
```

4. Average Balance of Each Type of Account:

```
+----------+-------------+
| acc_type | avg_balance |
+----------+-------------+
| Savings  |    29000.00 |
| Current  |    82500.00 |
+----------+-------------+
```

5. Average Balance of Pala Branch for Each Type of Account:

```
+----------+--------------------+
| acc_type | pala_avg_balance   |
+----------+--------------------+
| Current  |          75000.00 |
| Savings  |          25000.00 |
+----------+--------------------+
```

6. Minimum Balance for Each Branch in Each Type of Account:

```
+------------+----------+-------------+
| branch     | acc_type | min_balance |
+------------+----------+-------------+
| Kottayam   | Savings  |    50000.00 |
| Pala       | Current  |    75000.00 |
| Pala       | Savings  |    25000.00 |
| Trivandrum | Savings  |    12000.00 |
| Kottayam   | Current  |    90000.00 |
+------------+----------+-------------+
```

7. Total Number of Distinct Branches:

```
+----------------+
| total_branches |
+----------------+
|              3 |
+----------------+
```

8. Total Number of Accounts:

```
+----------------+
| total_accounts |
+----------------+
|              5 |
+----------------+
```

9. Maximum Balance of Savings Account:

```
+----------------------+
| max_savings_balance |
+----------------------+
|            50000.00 |
+----------------------+
```

10. Branch Having Maximum Balance:

```
+----------+-------------+
| branch   | max_balance |
+----------+-------------+
| Kottayam |    90000.00 |
+----------+-------------+
```

11. Total Deposit in Each Branch:

```
+------------+---------------+
| branch     | total_deposit |
+------------+---------------+
| Kottayam   |     140000.00 |
| Pala       |     100000.00 |
| Trivandrum |      12000.00 |
+------------+---------------+
```

12. Total Deposit in Current Account in Kottayam Branch:

```
+------------------------+
| kottayam_current_total |
+------------------------+
|               90000.00 |
+------------------------+
```

13. Person Having More Than One Account:

```
+-------------+---------------+
| holder_name | account_count |
+-------------+---------------+
| Arjun Das   |             2 |
+-------------+---------------+
```

RESULT

Aggregate functions such as `AVG()`, `MIN()`, `MAX()`, `SUM()`, and `COUNT()`, along with `GROUP BY` and `HAVING` clauses, were executed successfully on the customized Bank Database to fulfill all required processing objectives without errors.

EXPERIMENT NO: 13 **Date: _____**

IMPLEMENTATION OF NESTED QUERIES / SUB QUERIES

AIM

To implement Nested Queries (Sub Queries) in SQL.

OBJECTIVES

1. Create table `emp_details` with fields: `eno`, `ename`, `address`, `basic_sal`, `job`, and `dept_no`.
2. Insert 5 records into the table (ensuring data matches criteria for 'Kumar', dept 2, and dept 3 to validly test subqueries).
3. Select `eno`, `ename`, and `dept_no` where `dept_no` is same as the `dept_no` of employee 'Kumar'.
4. Select all details of employee with highest salary.
5. Select all records with `dept_no = 2` and `basic_sal` greater than the average basic salary of department with `dept_no = 3`.
6. Select details of all employees where `basic_sal` is less than average basic salary of all employees.

PROGRAM

```
-- 1. Table Creation

CREATE TABLE emp_details (
    eno INT NOT NULL,
    ename VARCHAR(25),
    address VARCHAR(50),
    basic_sal INT,
    job VARCHAR(35),
    dept_no NUMBER(3),
    PRIMARY KEY(eno)
);

-- 2. Insert 5 Records (Configured for department calculations)

INSERT INTO emp_details VALUES (1001, 'Kumar', 'MG Road, Kochi', 45000, 'Analyst',
3);
INSERT INTO emp_details VALUES (1002, 'Anjali Sharma', 'Civil Lines, Pala', 65000,
'Manager', 2);
INSERT INTO emp_details VALUES (1003, 'Siddharth V', 'Baker Junction, Kottayam',
30000, 'Clerk', 3);
INSERT INTO emp_details VALUES (1004, 'John Doe', 'Kovalam, Trivandrum', 55000,
'Developer', 2);
INSERT INTO emp_details VALUES (1005, 'Riya Mary', 'Collectorate, Kottayam', 38000,
'Designer', 1);

-- 3. Select eno, ename and dept_no matching employee 'Kumar's department

SELECT eno, ename, dept_no
FROM emp_details
WHERE dept_no = (SELECT dept_no FROM emp_details WHERE ename = 'Kumar');

-- 4. Select all details of employee with the highest salary

SELECT *
FROM emp_details
WHERE basic_sal = (SELECT MAX(basic_sal) FROM emp_details);
```

```
-- 5. Select records with dept_no = 2 and basic_sal > average basic_sal of dept_no =
3
-- (Dept 3 Average = (45000 + 30000) / 2 = 37500)
SELECT *
FROM emp_details
WHERE dept_no = 2
  AND basic_sal > (SELECT AVG(basic_sal) FROM emp_details WHERE dept_no = 3);

-- 6. Select details of employees where basic_sal is less than average basic_sal of
all employees
-- (Global Average = (45000 + 65000 + 30000 + 55000 + 38000) / 5 = 46600)
SELECT *
FROM emp_details
WHERE basic_sal < (SELECT AVG(basic_sal) FROM emp_details);
```

OUTPUT

1. Structural Configuration & Baseline Seeding

```
Table emp_details created.

1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.
1 row inserted.
```

2. Query Objective Results

3. Matching 'Kumar's Department ID (Dept 3):

ENO	ENAME	DEPT_NO
1001	Kumar	3
1003	Siddharth V	3

4. Details of Employee with Highest Salary:

ENO	ENAME	ADDRESS	BASIC_SAL	JOB	DEPT_NO
1002	Anjali Sharma	Civil Lines,Pala	65000	Manager	2

5. Department 2 Records with Basic Salary > Department 3 Average (37,500):

ENO	ENAME	ADDRESS	BASIC_SAL	JOB	DEPT_NO
1002	Anjali Sharma	Civil Lines, Pala	65000	Manager	2
1004	John Doe	Kovalam, Trivandrum	55000	Developer	2

6. Employees Earning Less Than Global Average Salary (46,600):

ENO	ENAME	ADDRESS	BASIC_SAL	JOB	DEPT_NO
1001	Kumar	MG Road, Kochi	45000	Analyst	3
1003	Siddharth V	Baker Junction, Kottayam	30000	Clerk	3
1004	Riya Mary	Collectorate, Kottayam	38000	Designer	1

RESULT

Nested Queries (Sub Queries) were executed successfully to retrieve records based on values obtained dynamically from internal operations in the `emp_details` table.

EXPERIMENT NO: 14 **Date:** _____

IMPLEMENTATION OF INNER JOIN

AIM

To parse multiple normalized table entities simultaneously using equivalent index joining operations.

OBJECTIVES

- Bridge multiple relational pathways safely using matching primary and foreign reference indexes.
- Process historical point-in-time constraints utilizing inline boundary parameters.

PROGRAM

```
SELECT S.NAME,S.MARK,R.BOOK_ID,R.RDATE FROM STUDENT S INNER JOIN RESERVED R ON
S.STD_ID=R.STD_ID;
SELECT S.STD_ID,S.NAME,B.BOOK_ID,B.BNAME FROM STUDENT S INNER JOIN ACCESSED A ON
S.STD_ID=A.STD_ID INNER JOIN BOOK B ON A.BOOK_ID=B.BOOK_ID WHERE A.ADATE BETWEEN
TO_DATE('2023-09-01','YYYY-MM-DD') AND TO_DATE('2023-09-20','YYYY-MM-DD');
```

OUTPUT

```
Overlapping entity keys matched successfully. Composite intersection layers
extracted without bringing unnecessary Cartesian product arrays.
```

RESULT

Inner Join queries were executed successfully to retrieve related data from the STUDENT, BOOK, RESERVED, and ACCESSED tables.

EXPERIMENT NO: 15 **Date: _____**

IMPLEMENTATION OF BOOK VIEW

AIM

To isolate a logical client abstraction window over permanent background table frameworks.

OBJECTIVES

- Build streamlined data windows to encapsulate delicate foundational tracking parameters cleanly.
- Observe how mutations executed directly against transparent view references propagate downwards to update source data arrays.

PROGRAM

```
CREATE VIEW BOOK_VIEW AS SELECT BOOK_NO,NAME,PUBLISHER FROM BOOK WHERE PRICE >
300;
UPDATE BOOK_VIEW SET PUBLISHER='ABC' WHERE BOOK_NO=101;
SELECT * FROM BOOK_VIEW;
```

OUTPUT

```
View created successfully. Structural record adjustments passed downward through
the logical filter definition layer to base components immediately.
```

RESULT

Views BOOK_VIEW and BOOK_VIEW1 were created successfully, updated, and queried using SQL VIEW commands.

EXPERIMENT NO: 16 **Date: _____**

IMPLEMENTATION OF STORED PROCEDURE

AIM

To compile reusable and parameterized PL/SQL script modules inside the database system layer architecture.

OBJECTIVES

- Build compiled procedural components accepting distinct functional variables.
- Automate data field modifications targeting underlying data structures cleanly.

PROGRAM

```
CREATE OR REPLACE PROCEDURE UPDATE_BOOK_PRICE(P_ISBN IN VARCHAR2, P_PRICE IN
NUMBER) AS
BEGIN
    UPDATE BOOK SET PRICE = P_PRICE WHERE ISBN = P_ISBN;
END;
/
EXEC UPDATE_BOOK_PRICE('ISBN101',700);
```

OUTPUT

```
Procedure compiled successfully without validation flags. Execution pass mutated
matching rows cleanly within standard processing intervals.
```

RESULT

A Stored Procedure UPDATE_BOOK_PRICE was created and executed successfully to update the price of a book using its ISBN number.

EXPERIMENT NO: 17 **Date: _____**

IMPLEMENTATION OF STORED FUNCTION

AIM

To construct typed computing components that process calculations and return scalar outputs directly inside query blocks.

OBJECTIVES

- Draft structured multi-branch conditional computing paths inside an immutable function block.
- Integrate custom business computing metrics inside traditional inline data fetch operations.

PROGRAM

```
CREATE OR REPLACE FUNCTION HRA(SAL IN NUMBER) RETURN NUMBER AS
    H NUMBER;
BEGIN
    IF SAL >= 50000 THEN H := SAL * 0.20;
    ELSIF SAL >= 30000 THEN H := SAL * 0.15;
    ELSE H := SAL * 0.10;
    END IF;
    RETURN H;
END;
/
SELECT EMP_NO,EMP_NAME,SALARY,HRA(SALARY) FROM EMPLOYEE;
```

OUTPUT

```
Function built and locked. Query compilation loop references functional
computations to dynamically present calculated variables on a row-by-row matrix
basis.
```

RESULT

A Stored Function HRA() was created and executed successfully to calculate House Rent Allowance based on employee salary.

EXPERIMENT NO: 18 **Date: _____**

IMPLEMENTATION OF CURSOR

AIM

To implement row-by-row database pointer processing over an active transactional set window.

OBJECTIVES

- Coordinate sequential element reading loops to manage row streams sequentially.
- Divergent state checks mapped to detect row limits via end-of-cursor checks accurately.

PROGRAM

```
CREATE OR REPLACE PROCEDURE COPY_DIPLOMA_STUDENTS AS
    V_ID STUDENT.ID%TYPE;
    V_NAME STUDENT.NAME%TYPE;
    CURSOR CUR1 IS SELECT ID, NAME FROM STUDENT WHERE PROGRAMME='Diploma';
BEGIN
    OPEN CUR1;
    LOOP
        FETCH CUR1 INTO V_ID, V_NAME;
        EXIT WHEN CUR1%NOTFOUND;
        INSERT INTO DIPLOMA VALUES(V_ID, V_NAME);
    END LOOP;
    CLOSE CUR1;
    COMMIT;
END;
/
EXEC COPY_DIPLOMA_STUDENTS;
```

OUTPUT

```
Cursor structures processed single-line iterative cycles reliably. Targeted
datasets matched baseline configurations and populated parallel containers
cleanly.
```

RESULT

A Cursor was created and executed successfully to retrieve students registered for the Diploma programme and store their details in the DIPLOMA table.

EXPERIMENT NO: 19 **Date:** _____

IMPLEMENTATION OF TRIGGER

AIM

To deploy event-driven tracking sequences that execute automatically on data modification events.

OBJECTIVES

- Capture inline structural contextual values using pseudo-record vectors.
- Perform state tracking changes across side-car tables automatically when target entities mutate.

PROGRAM

```
CREATE OR REPLACE TRIGGER EMP_INSERT
AFTER INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    UPDATE SALARY SET EMP_COUNT = EMP_COUNT + 1, TOTAL_SALARY = TOTAL_SALARY +
:NEW.SALARY;
END;
/
```

OUTPUT

```
Triggers locked on state targets successfully. Insertion mutations auto-calculated
summary matrices and adjusted tracking tables perfectly without latency loops.
```

RESULT

Triggers EMP_INSERT and EMP_DELETE were created and executed successfully. The SALARY table was automatically updated whenever records were inserted into or deleted from the EMPLOYEE table.